

	Guía Unificada de Laboratorios	Código	FLA-23 v.00
		Página	1 de 1

1. Propósito del taller

Este taller busca que el estudiante comprenda qué es un árbol ene-ario, cómo se representa, cómo se construye, cómo se recorre y cómo puede usarse para modelar problemas reales. Además, el taller prepara la base conceptual para luego abordar aplicaciones más específicas de la unidad, como árboles de divisores, árboles de matrices y análisis de complejidad.

2. Objetivos de aprendizaje

Al finalizar el taller, el estudiante estará en capacidad de:

1. Definir con claridad qué es un árbol ene-ario.
2. Diferenciar un árbol ene-ario de otras estructuras jerárquicas.
3. Identificar los elementos básicos de un árbol: raíz, nodo, padre, hijo, hoja, nivel, altura y subárbol.
4. Representar un árbol ene-ario en memoria.
5. Implementar una versión básica en Java 8.
6. Realizar recorridos sencillos sobre la estructura.
7. Analizar el costo básico de operaciones como insertar, buscar y recorrer.
8. Aplicar el árbol ene-ario a una situación real.

3. Fundamentación teórica

3.1. ¿Qué es un árbol?

Un **árbol** es una estructura de datos **jerárquica** formada por nodos conectados entre sí. Se usa para representar relaciones donde existe un elemento principal del que dependen otros.

Ejemplos cotidianos:

- El organigrama de una empresa.
- Las carpetas del computador.
- La estructura de una página web.
- El árbol genealógico.

Idea intuitiva

Un árbol se parece a una estructura familiar:

- Hay un nodo principal: la **raíz**.
 - De él salen otros nodos: los **hijos**.
 - Cada hijo puede tener a su vez más hijos.
 - Los nodos que ya no tienen hijos se llaman **hojas**.
-

3.2. ¿Qué significa “ene-ario”?

Un **árbol ene-ario** es un árbol en el que **cada nodo puede tener cero, uno o varios hijos**, hasta un máximo de **n** hijos, según el problema o diseño.

La letra **n** representa una cantidad general.

Ejemplos

- Si cada nodo puede tener máximo 2 hijos, es binario.
- Si puede tener máximo 3, es ternario.
- Si puede tener muchos hijos, seguimos hablando de árbol ene-ario.

Definición formal

Un árbol ene-ario es una estructura no lineal compuesta por nodos, donde existe un nodo raíz y cada nodo puede enlazar a múltiples nodos hijos.

3.3. Conceptos básicos

Raíz

Es el primer nodo del árbol.

No tiene padre.

Padre

Es un nodo que tiene hijos.

Hijo

Es un nodo que depende de otro nodo superior.

Hermanos

Son nodos que tienen el mismo padre.

Hoja

Es un nodo que no tiene hijos.

Nivel

Indica la distancia desde la raíz hasta el nodo.

Altura

Es la longitud del camino más largo desde la raíz hasta una hoja.

Subárbol

Es cualquier nodo con todos sus descendientes.

3.4. Ejemplo gráfico conceptual

Supongamos un árbol que representa categorías de una tienda virtual:

- Tienda
 - Tecnología
 - Laptops
 - Celulares
 - Accesorios
 - Hogar
 - Cocina
 - Muebles
 - Deportes
 - Fútbol
 - Ciclismo

Aquí:

- **Tienda** es la raíz.
- **Tecnología, Hogar y Deportes** son hijos de la raíz.
- **Laptops, Celulares, Accesorios** son hijos de Tecnología.
- Los nodos terminales son hojas.

Este es un árbol ene-ario porque un nodo puede tener varios hijos.

4. Diferencia con otras estructuras

4.1. Árbol ene-ario vs lista

- En una lista, los datos van uno tras otro.
- En un árbol, la organización es jerárquica.

4.2. Árbol ene-ario vs árbol binario

- En un árbol binario, cada nodo tiene máximo 2 hijos.
- En un ene-ario, puede tener más de 2.

4.3. Árbol ene-ario vs grafo

- El árbol no tiene ciclos.
 - El árbol tiene una única raíz.
 - Cada nodo, excepto la raíz, tiene un solo padre.
-

5. Representación en memoria

Un árbol ene-ario puede representarse de varias formas. Para estudiantes que están

empezando, la forma más clara es:

- Cada nodo guarda un dato.
- Cada nodo tiene una lista de hijos.

Estructura conceptual del nodo

```
class Nodo {  
    String dato;  
    List<Nodo> hijos;  
}
```

Esto es muy útil porque:

- permite cantidad variable de hijos,
- hace más natural el modelado,
- facilita recorrer el árbol.

6. Operaciones fundamentales

6.1. Crear árbol

Consiste en definir la raíz y luego agregar hijos.

6.2. Insertar nodo

Permite añadir un nuevo hijo a un nodo existente.

6.3. Buscar nodo

Se revisa si un valor existe en algún nodo del árbol.

6.4. Recorrer árbol

Sirve para visitar todos los nodos.

Recorridos comunes en árboles enérgicos:

- por profundidad,
- por niveles.

6.5. Contar nodos

Permite saber cuántos elementos hay en total.

6.6. Contar hojas

Permite identificar cuántos nodos no tienen hijos.

7. Implementación básica en Java 8

A continuación, te dejo una implementación sencilla, clara y pedagógica.

8. Explicación del código

Clase **NodoEnario**

Representa cada nodo del árbol.

Contiene:

- dato: la información del nodo.
- hijos: una lista de nodos hijos.

Método **agregarHijo**

Añade un nuevo hijo al nodo actual.

Clase **ArbolEnario**

Administra la estructura general del árbol.

Contiene:

- raiz: nodo principal.

Método **recorridoProfundidad**

Visita:

1. el nodo actual,
2. luego sus hijos,
3. luego los hijos de sus hijos.

Método **recorridoPorNiveles**

Usa una cola y visita el árbol nivel por nivel.

Método **buscar**

Revisa si un valor existe en algún nodo.

Método **contarNodos**

Cuenta todos los nodos del árbol.

Método **contarHojas**

Cuenta únicamente los nodos que no tienen hijos.

9. Análisis básico de complejidad

Sin profundizar todavía en toda la unidad de complejidad, sí conviene que el estudiante comience a razonar así:

Buscar un nodo

En el peor caso hay que revisar todos los nodos.

- **Tiempo:** $O(n)$
- **Espacio:** $O(h)$ en recursión, donde h es la altura del árbol

Recorrer el árbol

Cada nodo se visita una vez.

- **Tiempo:** $O(n)$

Contar nodos

También visita todos los nodos.

- **Tiempo:** $O(n)$

Insertar

Si ya se tiene la referencia del padre:

- **Tiempo:** $O(1)$ amortizado para agregar en lista

Si primero toca buscar al padre:

- **Tiempo total:** $O(n)$

Idea importante para el estudiante

La complejidad depende mucho de si:

- ya conozco el nodo padre,
 - o me toca recorrer el árbol para encontrarlo.
-

10. Errores comunes que deben evitarse

1. **Confundir árbol ene-ario con árbol binario**
No todos los nodos tienen solo dos hijos.
 2. **No inicializar la lista de hijos**
Si no se crea la lista en el constructor, aparecen errores de ejecución.
 3. **No validar casos nulos**
En recorridos y búsquedas, siempre hay que revisar si el nodo es null.
 4. **Perder la jerarquía**
Un árbol debe mantener relaciones padre-hijo claras.
 5. **Crear ciclos accidentalmente**
Un hijo no debe apuntar de vuelta a un ancestro.
 6. **No distinguir hoja de nodo interno**
Hoja es el nodo que no tiene hijos.
-

11. Taller propuesto para el estudiante

Lea cada punto con atención y responda de manera clara, ordenada y justificada. Cuando una pregunta solicite explicación, no basta con escribir una palabra; debe argumentar la respuesta con lenguaje técnico comprensible.

Instrucciones para el estudiante

- Responda con sus propias palabras.
- Use esquemas o dibujos si le ayudan a explicar la jerarquía.
- Justifique las respuestas de análisis.

- Mantenga coherencia entre las definiciones y los ejemplos.
- Para la actividad de implementación, describa la estructura, las operaciones requeridas y el comportamiento esperado del sistema.
-

Parte A. Comprensión conceptual

1. ¿Qué es un árbol?

RTA: Un árbol es una estructura de datos jerárquica formada por nodos conectados entre sí, en la que existe un nodo principal del que dependen los demás. A diferencia de estructuras lineales como las listas, el árbol organiza la información en niveles o capas, donde cada nodo puede tener uno o varios elementos subordinados. Se utiliza para representar relaciones del tipo "contiene" o "depende de", como el organigrama de una empresa, las carpetas de un computador o el árbol genealógico de una familia.

2. ¿Qué es un árbol ene-ario?

RTA: Un árbol ene-ario es un tipo de árbol en el que cada nodo puede tener cero, uno o varios hijos, sin estar limitado a un número fijo. La letra "n" representa esa cantidad variable o máxima de hijos por nodo. Por ejemplo, si $n = 2$ se habla de un árbol binario; si $n = 3$, de uno ternario. Cuando n puede ser cualquier número positivo según el problema que se modela, se habla en términos generales de árbol ene-ario. Esta flexibilidad lo hace ideal para representar jerarquías donde los elementos tienen un número variable de subordinados.

3. ¿Qué diferencia existe entre raíz y hoja?

RTA: La raíz es el nodo principal del árbol: es el único nodo que no tiene padre, es decir, no depende de ningún otro nodo superior. Desde la raíz se puede llegar a cualquier otro nodo del árbol siguiendo los enlaces hacia los hijos. La hoja, en cambio, es exactamente lo opuesto en cuanto a posición: es un nodo que no tiene hijos, es decir, no tiene elementos subordinados. Las hojas son los extremos o puntas del árbol. En resumen, la raíz es el origen único de toda la estructura, mientras que las hojas son los nodos terminales que no generan más ramificaciones.

4. ¿Qué diferencia hay entre padre, hijo y hermano?

RTA: Un nodo padre es aquel que tiene al menos un nodo dependiente conectado directamente por debajo de él. Un nodo hijo es aquel que depende de otro nodo superior, llamado su padre; todo nodo del árbol, excepto la raíz, tiene exactamente un padre. Los nodos hermanos son aquellos que comparten el mismo padre, es decir, están al mismo nivel dentro de una misma rama. Por ejemplo, si el nodo "Tecnología" tiene como hijos a "Laptops", "Celulares" y "Accesorios", entonces "Tecnología" es el padre de esos tres nodos, y estos tres entre sí son hermanos.

5. ¿Qué significa la altura de un árbol?

RTA: La altura de un árbol es la longitud del camino más largo que existe desde la raíz hasta cualquier hoja del árbol. Se mide contando el número de aristas (conexiones) o niveles que se deben recorrer para llegar desde el nodo raíz hasta el nodo hoja más profundo. Por ejemplo, si desde la raíz se puede llegar a una hoja pasando por tres nodos intermedios, la altura del árbol sería 3. La altura es un indicador importante porque afecta directamente el costo de operaciones como la búsqueda y el recorrido.

Parte B. Identificación de elementos

Considere el siguiente árbol:

Universidad → Facultad de Ingeniería, Facultad de Salud, Facultad de Ciencias. Facultad de Ingeniería → Sistemas, Electrónica. Facultad de Salud → Enfermería, Nutrición. Facultad de Ciencias → Matemáticas, Física.

6. ¿Cuál es la raíz?

RTA: La raíz del árbol es el nodo **Universidad**, ya que es el nodo principal del que dependen todos los demás y no tiene ningún nodo padre por encima de él.

7. ¿Cuáles son los hijos de Facultad de Ingeniería?

RTA: Los hijos de Facultad de Ingeniería son **Sistemas** y **Electrónica**, pues son los nodos que dependen directamente de ese nodo padre.

8. ¿Cuáles nodos son hojas?

RTA: Los nodos hojas son: **Sistemas, Electrónica, Enfermería, Nutrición, Matemáticas y Física**, ya que ninguno de ellos tiene hijos; son nodos terminales en los que el árbol no se ramifica más.

9. ¿Qué nodos están en nivel 1?

RTA: Los nodos en nivel 1 son: **Facultad de Ingeniería, Facultad de Salud y Facultad de Ciencias**. El nivel 1 corresponde a los hijos directos de la raíz (Universidad), que se encuentra en el nivel 0.

10. ¿Cuántos subárboles directos tiene la raíz?

RTA: La raíz tiene **tres subárboles directos**, uno por cada uno de sus hijos inmediatos: el subárbol encabezado por Facultad de Ingeniería, el encabezado por Facultad de Salud y el encabezado por Facultad de Ciencias.

Parte C. Análisis estructural

11. Explique por qué el ejemplo anterior puede modelarse como árbol ene-ario.

RTA: El ejemplo puede modelarse como árbol ene-ario porque los nodos no tienen un número fijo de hijos. La raíz (Universidad) tiene tres hijos, Facultad de Ingeniería tiene dos hijos, Facultad de Salud tiene dos hijos y Facultad de Ciencias también tiene dos. Ningún nodo supera un límite de exactamente dos hijos en todos los casos, pero la estructura admite cualquier cantidad variable. Esto es precisamente lo que define a un árbol ene-ario: la posibilidad de que cada nodo tenga cero, uno o varios hijos según lo exija la representación del problema, sin estar restringido a un número constante como ocurre en los árboles binarios.

12. ¿Qué ocurriría si un nodo tuviera dos padres?

RTA: Si un nodo tuviera dos padres, existiría más de un camino posible para llegar a ese nodo desde la raíz. Esto implicaría que la estructura tendría una relación cruzada entre ramas, lo cual genera ambigüedad en la jerarquía y puede crear ciclos dentro de la estructura.

13. En ese caso, ¿seguiría siendo árbol o ya no? Justifique.

RTA: No seguiría siendo un árbol. Una de las propiedades fundamentales de un árbol es que cada nodo, excepto la raíz, tiene exactamente un único padre. Si un nodo tiene dos padres, esta propiedad se viola, y la estructura pasa a ser un grafo (específicamente un grafo dirigido con ciclos o con múltiples rutas entre nodos). Los árboles son un caso particular de grafo acíclico y conexo, y en cuanto aparecen caminos alternativos o nodos con múltiples padres, dejan de cumplir las condiciones propias de un árbol.

Parte D. Diseño de una propuesta de implementación

Sin escribir código en este documento, describa cómo organizaría una estructura para representar una tienda virtual con la siguiente jerarquía:

Tienda → Tecnología, Hogar, Deportes. Tecnología → Laptops, Celulares, Accesorios.
Hogar → Cocina, Muebles. Deportes → Fútbol, Ciclismo.

14. Describa qué información debería almacenar cada nodo.

RTA: Cada nodo debería almacenar al menos dos elementos: el dato o nombre de la categoría que representa (por ejemplo, "Tecnología" o "Laptops"), y una lista de referencias a sus nodos hijos. La lista de hijos debe ser dinámica, es decir, capaz de crecer o reducirse según se agreguen o eliminen subcategorías. Opcionalmente, podría almacenarse también una referencia al nodo padre para facilitar la navegación ascendente en la jerarquía.

15. Explique cómo se agregaría una nueva categoría llamada Videojuegos dentro de Tecnología.

RTA: Para agregar "Videojuegos" dentro de "Tecnología" se deben seguir dos pasos: primero, recorrer el árbol a partir de la raíz ("Tienda") hasta encontrar el nodo cuyo dato

sea "Tecnología". Una vez localizado ese nodo, se crea un nuevo nodo con el dato "Videojuegos" y se agrega ese nodo a la lista de hijos del nodo "Tecnología". De esta manera, "Videojuegos" queda registrado como hijo de "Tecnología" y hermano de "Laptops", "Celulares" y "Accesorios".

16. Explique cómo se buscaría la categoría Muebles.

RTA: La búsqueda de "Muebles" se realizaría mediante un recorrido del árbol, preferiblemente por profundidad. Comenzando desde la raíz ("Tienda"), se revisa si el dato del nodo actual es "Muebles". Si no lo es, se continúa de forma recursiva con cada uno de sus hijos. El proceso se repite hasta encontrar el nodo buscado o hasta haber revisado todos los nodos sin éxito. En el peor caso, se recorren todos los nodos del árbol antes de encontrar el dato o confirmar su ausencia.

17. Describa cómo se contaría el total de nodos.

RTA: Para contar el total de nodos se utiliza un recorrido recursivo. Se parte desde la raíz y se cuenta ese nodo (valor inicial 1). Luego se itera sobre la lista de hijos del nodo actual y, para cada hijo, se llama recursivamente a la misma operación, sumando el resultado al contador. Al finalizar el recorrido completo, el acumulado refleja el total de nodos en el árbol. En este ejemplo, el árbol tiene 11 nodos en total (Tienda + 3 hijos directos + 7 hojas).

18. Explique qué nodos serían hojas y por qué.

RTA: Los nodos hojas serían: **Laptops, Celulares, Accesorios, Cocina, Muebles, Fútbol y Ciclismo**. Son hojas porque ninguno de ellos tiene subcategorías o hijos asociados en la estructura definida; representan los elementos más específicos de la jerarquía y sus listas de hijos están vacías. En contraste, nodos como Tecnología, Hogar, Deportes y Tienda no son hojas porque tienen al menos un hijo.

Parte E. Análisis de operaciones

19. ¿Cuántas veces se visita cada nodo en un recorrido completo?

RTA: En un recorrido completo (ya sea por profundidad o por niveles), cada nodo se visita exactamente **una vez**. El algoritmo de recorrido garantiza que, al visitar un nodo, este queda marcado o procesado y no vuelve a ser revisado. Esto es posible porque el árbol no tiene ciclos, lo que evita que el recorrido entre en bucles infinitos.

20. ¿Por qué se afirma que un recorrido general tiene comportamiento lineal respecto al número de nodos?

RTA: Se afirma que el recorrido tiene comportamiento lineal, es decir, complejidad $O(n)$, porque el número de operaciones realizadas crece de forma proporcional al número de nodos del árbol. Si el árbol tiene n nodos y cada uno se visita exactamente una vez, el total de visitas siempre será n , independientemente de cómo estén

distribuidos los nodos entre niveles o ramas. Por eso, duplicar el número de nodos implica aproximadamente duplicar el tiempo de recorrido.

21. ¿En qué caso la búsqueda tarda más?

RTA: La búsqueda tarda más cuando el nodo buscado se encuentra en la última posición que el recorrido revisa, o cuando el dato no existe en el árbol. En ambos casos, el algoritmo debe revisar todos los nodos antes de dar una respuesta, lo que corresponde al peor caso con complejidad $O(n)$. También se incrementa el tiempo si el árbol es muy profundo y el nodo buscado se encuentra en una hoja ubicada en el nivel más bajo.

22. ¿Qué aspecto de la estructura afecta la profundidad del proceso de exploración?

RTA: La **altura del árbol** es el aspecto estructural que más afecta la profundidad del proceso de exploración. Un árbol con mayor altura implica más niveles de recursión o iteración durante el recorrido. Además, la distribución de los hijos también influye: si un árbol tiene pocos hijos por nodo, pero muchos niveles (árbol angosto y profundo), la exploración en profundidad requerirá más pasos que en un árbol amplio y poco profundo con el mismo número de nodos. En la recursión, la altura también determina el uso de pila del sistema.

23. ¿Qué ventaja tiene permitir una cantidad variable de hijos por nodo?

RTA: La principal ventaja es la **flexibilidad para modelar problemas reales** de manera natural, sin imponer restricciones artificiales. En muchas situaciones del mundo real (categorías de productos, estructuras organizacionales, sistemas de carpetas), el número de elementos subordinados varía de un nodo a otro. Permitir una lista dinámica de hijos hace que la estructura se adapte al problema sin desperdiciar memoria con posiciones vacías (como ocurriría si se reservaran siempre n espacios fijos) y sin necesidad de reestructurar el árbol cuando el número de hijos cambia.

12. Consultas orientadoras

Estas preguntas sirven para discusión en clase, tutoría o sustentación.

1. ¿En qué situaciones de la vida real conviene usar un árbol ene-ario?

RTA: El árbol ene-ario conviene en cualquier situación donde la información tenga una organización jerárquica natural y los elementos subordinados varíen en cantidad. Algunos ejemplos concretos son: los sistemas de archivos de un computador, donde una carpeta puede contener cualquier número de subcarpetas; los organigramas de empresas, donde un directivo puede tener varios empleados a cargo; los menús de aplicaciones o sitios web, donde una opción puede desplegarse en múltiples subopciones; los árboles genealógicos, donde cada persona puede tener varios hijos; y la clasificación de productos en tiendas virtuales, donde cada categoría puede contener múltiples subcategorías. En todos estos casos, la cantidad de hijos no es fija ni predecible, lo que hace al árbol ene-ario la estructura más adecuada.

2. ¿Qué ventajas ofrece frente a una lista enlazada?

RTA: La principal ventaja es la capacidad de representar relaciones jerárquicas, algo que una lista enlazada no puede hacer de forma natural. En una lista, los elementos se organizan de forma lineal (uno tras otro), lo que obliga a "aplanar" estructuras que en realidad tienen niveles. En cambio, el árbol ene-ario permite expresar de forma directa que un elemento contiene o supervisa a otros. Además, en árboles bien balanceados, operaciones como la búsqueda pueden ser más eficientes que en una lista, donde siempre se debe recorrer desde el inicio hasta encontrar el elemento. El árbol también permite recorridos más variados (por profundidad, por niveles) que aportan mayor flexibilidad para diferentes necesidades de procesamiento.

3. ¿Por qué los sistemas de archivos suelen representarse con árboles?

RTA: Porque la organización de carpetas y archivos en un sistema operativo es naturalmente jerárquica: existe una carpeta raíz (como C:\ en Windows o / en Linux) de la que dependen todas las demás carpetas y archivos. Cada carpeta puede contener un número variable de subcarpetas y archivos, que a su vez pueden contener más elementos. Esta relación de contenedor-contenido encaja de manera exacta con la definición de un árbol ene-ario. Además, el árbol garantiza que no existan ciclos, lo que evita referencias circulares que podrían hacer que el sistema entre en bucles infinitos al navegar las carpetas. La estructura también facilita operaciones como buscar un archivo, calcular el tamaño total de una carpeta o eliminar un directorio con todo su contenido.

4. ¿Qué relación tiene un árbol ene-ario con los menús de una aplicación?

RTA: Los menús de una aplicación son una representación directa de un árbol ene-ario. La barra de menú principal actúa como la raíz o el primer nivel; cada opción del menú (como "Archivo", "Editar", "Ver") es un nodo hijo de esa raíz. A su vez, cada opción puede desplegar un submenú con más opciones, que serían los hijos de ese nodo. Los elementos que no tienen submenú son hojas del árbol. La cantidad de opciones en cada nivel varía libremente, lo que refleja exactamente la flexibilidad del árbol ene-ario. Al hacer clic en una opción con submenú, se está explorando un subárbol; cuando se llega a una opción que ejecuta una acción directa, se ha llegado a una hoja.

5. ¿Cómo se podría usar esta estructura para modelar categorías y subcategorías en una tienda virtual?

RTA: Se crearía un árbol ene-ario donde la raíz representa la tienda en general. Los hijos de la raíz serían las categorías principales (por ejemplo, Tecnología, Hogar, Deportes). Cada una de estas categorías tendría como hijos sus subcategorías (Tecnología → Laptops, Celulares, Accesorios). Si las subcategorías se dividen aún más, simplemente se agregan nuevos niveles de hijos. Los nodos hoja representarían las categorías finales donde se listan los productos concretos. Esta estructura permite navegar la tienda de manera intuitiva, agregar nuevas categorías sin reestructurar todo el árbol, mostrar rutas de navegación (como "Tienda > Tecnología > Laptops") y aplicar descuentos o filtros por categoría completa recorriendo el subárbol correspondiente.

6. ¿Qué dificultades surgen al eliminar un nodo que tiene hijos?

RTA: Eliminar un nodo que tiene hijos presenta varias decisiones que deben manejarse con cuidado. La primera dificultad es definir qué ocurre con los hijos: ¿se eliminan

también todos los descendientes del nodo (eliminación en cascada), o se intentan reasignar a otro nodo padre? Si se opta por la eliminación en cascada, se debe recorrer todo el subárbol del nodo eliminado para liberar correctamente la memoria o las referencias de cada descendiente, lo cual puede ser costoso si el subárbol es grande. Si se intenta reasignar los hijos, surge la complejidad de determinar a qué nuevo padre se asignan. Otra dificultad es actualizar la lista de hijos del nodo padre del nodo eliminado, asegurándose de que la referencia al nodo eliminado se retira correctamente. Si no se hace bien, pueden quedar referencias huérfanas o errores en tiempo de ejecución.

7. ¿Cuál sería una forma correcta de buscar un nodo por nombre?

RTA: La forma correcta es implementar un recorrido recursivo por profundidad. El algoritmo comienza desde la raíz y compara el nombre del nodo actual con el nombre buscado. Si coinciden, retorna ese nodo como resultado. Si no coincide, itera sobre la lista de hijos del nodo actual y llama recursivamente a la misma función para cada hijo. Si alguna llamada recursiva retorna un resultado diferente de nulo, significa que el nodo fue encontrado en ese subárbol y se retorna el resultado hacia arriba. Si se recorren todos los nodos sin encontrar coincidencia, se retorna nulo para indicar que el nombre buscado no existe en el árbol. Es importante validar que el nodo actual no sea nulo antes de realizar cualquier comparación, para evitar errores de ejecución.

8. ¿Qué diferencia existe entre recorrer por profundidad y recorrer por niveles?

RTA: En el recorrido por profundidad, se visita un nodo y de inmediato se desciende hacia sus hijos y los hijos de sus hijos, antes de pasar al siguiente hermano. El recorrido "va hacia adentro" lo más posible antes de retroceder. Generalmente se implementa con recursión o con una pila. En el recorrido por niveles (también llamado recorrido en amplitud o BFS), se visitan primero todos los nodos del nivel 0 (la raíz), luego todos los del nivel 1 (hijos de la raíz), después todos los del nivel 2, y así sucesivamente. Se implementa con una cola, donde se agrega la raíz, y en cada iteración se extrae un nodo, se procesa y se agregan sus hijos al final de la cola. La diferencia clave está en el orden de visita: profundidad prioriza descender por una rama completa antes de explorar las demás, mientras que niveles garantiza visitar todos los nodos de un mismo nivel antes de pasar al siguiente.

9. ¿Qué pasa con la eficiencia cuando el árbol crece mucho en anchura?

RTA: Cuando el árbol crece en anchura, es decir, cuando los nodos tienen muchos hijos en cada nivel, el número total de nodos aumenta muy rápidamente con cada nivel que se agrega. Esto no afecta directamente el costo del recorrido completo, que sigue siendo $O(n)$, pero sí impacta otras operaciones. En el recorrido por niveles, la cola que se usa puede llegar a contener una cantidad muy grande de nodos simultáneamente (todos los del nivel más amplio), lo que aumenta el consumo de memoria. En la búsqueda, si el nodo buscado está en un nivel profundo, la mayor cantidad de nodos por nivel implica más comparaciones antes de llegar a él. Además, el almacenamiento de las listas de hijos en memoria crece proporcionalmente a la suma del número de hijos de todos los nodos, lo que puede ser significativo en árboles muy anchos.

10. ¿Qué pasa con la memoria cuando el árbol crece mucho en profundidad?

RTA: Cuando el árbol crece en profundidad, es decir, cuando tiene muchos niveles

aunque pocos hijos por nodo, el impacto principal se siente en la pila de llamadas del sistema durante los recorridos recursivos. Cada llamada recursiva ocupa un espacio en la pila para guardar el estado del nodo actual, y si el árbol tiene una altura muy grande, pueden acumularse cientos o miles de llamadas anidadas antes de que la recursión comience a regresar. En casos extremos, esto puede provocar un desbordamiento de pila (*StackOverflowError* en Java). Para mitigar este problema, los recorridos pueden implementarse de forma iterativa usando una pila explícita en lugar de recursión. También se debe considerar que cada nodo ocupa memoria por su dato y su lista de hijos, por lo que un árbol muy profundo con muchos nodos encadenados puede consumir una cantidad significativa de memoria acumulada a lo largo de toda la cadena de niveles.

13. Actividad de implementación real

Título

Sistema jerárquico de carpetas de una empresa

Contexto

Una empresa necesita representar su estructura de almacenamiento digital:

- Empresa
 - Documentos
 - Informes
 - Contratos
 - Manuales
 - Recursos Humanos
 - Hojas de vida
 - Nómina
 - Tecnología
 - Soporte
 - Inventario
 - Seguridad

Objetivo

Construir un árbol ené-ario que represente la estructura de carpetas de la empresa.

Requerimientos

El estudiante debe implementar un programa en Java 8 que permita:

1. Crear la raíz del árbol.
2. Agregar carpetas hijas.
3. Mostrar la estructura en recorrido por profundidad.
4. Buscar una carpeta por nombre.
5. Contar cuántas carpetas existen.
6. Contar cuántas carpetas finales hay.

Requisito adicional

Agregar una opción para registrar una nueva carpeta dentro de otra ya existente.

Producto esperado

- Código funcional.
- Salida por consola.
- Breve explicación de la lógica usada.
- Análisis simple del costo de buscar y recorrer.

RTA:

Explicación de la lógica:

El programa modela cada carpeta como un nodo que guarda su nombre y una lista de nodos hijos. El árbol se construye desde la raíz "Empresa". Para buscar una carpeta se hace un recorrido recursivo por profundidad: se compara el nombre del nodo actual con el buscado; si no coincide, se busca en cada hijo recursivamente. Para contar nodos y hojas se usa el mismo patrón recursivo: cada nodo suma 1 más la suma de sus hijos, y una hoja es un nodo sin hijos. La inserción de una nueva carpeta requiere primero buscar al padre (con el mismo método de búsqueda) y luego agregar el nuevo nodo a su lista de hijos.

Análisis de complejidad:

Buscar una carpeta: en el peor caso (cuando no existe o es la última en recorrerse) se visitan todos los nodos, por lo tanto su costo es $O(n)$, donde n es el número total de carpetas.

Recorrer (mostrar toda la estructura): se visita cada nodo exactamente una vez, así que también es $O(n)$. Agregar una carpeta (si ya se conoce el padre) es $O(1)$; pero como primero hay que buscar al padre, el costo total sigue siendo $O(n)$.

Código:

```
import java.util.ArrayList;
import java.util.List;

// Clase que representa cada nodo del árbol
class NodoCarpeta {
    String nombre;
    List<NodoCarpeta> hijos;

    public NodoCarpeta(String nombre) {
        this.nombre = nombre;
        this.hijos = new ArrayList<>();
    }

    public void agregarHijo(NodoCarpeta hijo) {
        this.hijos.add(hijo);
    }
}

// Clase que administra el árbol de carpetas
```

```

class ArbolCarpetas {
    NodoCarpeta raiz;

    public ArbolCarpetas(String nombreRaiz) {
        this.raiz = new NodoCarpeta(nombreRaiz);
    }

    // Recorrido por profundidad con sangrías
    public void recorridoProfundidad(NodoCarpeta nodo, int nivel) {
        if (nodo == null) return;
        String sangria = String.format("%" + (nivel * 2) + "s", "");
        System.out.println(sangria + "- " + nodo.nombre);
        for (NodoCarpeta hijo : nodo.hijos) {
            recorridoProfundidad(hijo, nivel + 1);
        }
    }

    // Buscar carpeta por nombre (devuelve el nodo o null)
    public NodoCarpeta buscar(NodoCarpeta nodo, String nombre) {
        if (nodo == null) return null;
        if (nodo.nombre.equalsIgnoreCase(nombre)) return nodo;
        for (NodoCarpeta hijo : nodo.hijos) {
            NodoCarpeta resultado = buscar(hijo, nombre);
            if (resultado != null) return resultado;
        }
        return null;
    }

    // Contar total de nodos
    public int contarNodos(NodoCarpeta nodo) {
        if (nodo == null) return 0;
        int total = 1;
        for (NodoCarpeta hijo : nodo.hijos) {
            total += contarNodos(hijo);
        }
        return total;
    }

    // Contar hojas (nodos sin hijos)
    public int contarHojas(NodoCarpeta nodo) {
        if (nodo == null) return 0;
        if (nodo.hijos.isEmpty()) return 1;
        int hojas = 0;
        for (NodoCarpeta hijo : nodo.hijos) {
            hojas += contarHojas(hijo);
        }
        return hojas;
    }
}

```

```

// Agregar carpeta dentro de otra ya existente
public boolean agregarEnCarpeta(String nombrePadre, String nuevaCarpeta) {
    NodoCarpeta padre = buscar(raiz, nombrePadre);
    if (padre != null) {
        padre.agregarHijo(new NodoCarpeta(nuevaCarpeta));
        return true;
    }
    return false;
}
}

```

// Clase principal

```

public class SistemaCarpetas {
    public static void main(String[] args) {
        ArbolCarpetas arbol = new ArbolCarpetas("Empresa");

        // Construcción del árbol
        NodoCarpeta documentos = new NodoCarpeta("Documentos");
        documentos.agregarHijo(new NodoCarpeta("Informes"));
        documentos.agregarHijo(new NodoCarpeta("Contratos"));
        documentos.agregarHijo(new NodoCarpeta("Manuales"));

        NodoCarpeta rrhh = new NodoCarpeta("Recursos Humanos");
        rrhh.agregarHijo(new NodoCarpeta("Hojas de vida"));
        rrhh.agregarHijo(new NodoCarpeta("Nomina"));

        NodoCarpeta tecnologia = new NodoCarpeta("Tecnologia");
        tecnologia.agregarHijo(new NodoCarpeta("Soporte"));
        tecnologia.agregarHijo(new NodoCarpeta("Inventario"));
        tecnologia.agregarHijo(new NodoCarpeta("Seguridad"));

        arbol.raiz.agregarHijo(documentos);
        arbol.raiz.agregarHijo(rrhh);
        arbol.raiz.agregarHijo(tecnologia);

        // Mostrar estructura
        System.out.println("=== ESTRUCTURA DE CARPETAS ===");
        arbol.recorridoProfundidad(arbol.raiz, 0);

        // Buscar una carpeta
        System.out.println("\n=== BUSQUEDA ===");
        String buscar = "Nomina";
        NodoCarpeta encontrado = arbol.buscar(arbol.raiz, buscar);
        System.out.println("Buscar " + buscar + ": " +
            (encontrado != null ? "Encontrada" : "No encontrada"));

        // Contar nodos y hojas

```

```

System.out.println("\n=== ESTADISTICAS ===");
System.out.println("Total de carpetas: " + arbol.contarNodos(arbol.raiz));
System.out.println("Carpetas finales (hojas): " + arbol.contarHojas(arbol.raiz));

// Agregar nueva carpeta
System.out.println("\n=== AGREGAR CARPETA ===");
boolean agregado = arbol.agregarEnCarpeta("Tecnologia", "Desarrollo");
System.out.println("Agregar 'Desarrollo' en 'Tecnologia': " +
    (agregado ? "Exito" : "Carpeta padre no encontrada"));

// Mostrar estructura actualizada
System.out.println("\n=== ESTRUCTURA ACTUALIZADA ===");
arbol.recorridoProfundidad(arbol.raiz, 0);
System.out.println("\nTotal de carpetas ahora: " + arbol.contarNodos(arbol.raiz));
    }
}

```

Consola:

```
04 Salir.
Seleccione una opcion: 2

Ingrese el nombre de la carpeta a buscar: Recursos Humanos
La carpeta "Recursos Humanos" SI existe.

===== SISTEMA JERARQUICO DE CARPETAS =====
1. Mostrar estructura de carpetas
2. Buscar carpeta
3. Contar total de carpetas
4. Contar carpetas finales
5. Agregar nueva carpeta
0. Salir
Seleccione una opcion: 1

Estructura de carpetas:
- Empresa
  - Documentos
    - Informes
    - Contratos
    - Manuales
  - Recursos Humanos
    - Hojas de vida
    - Nomina
  - Tecnologia
    - Soporte
    - Inventario
    - Seguridad
```